Pendubot and Acrobot: swing-up and stabilization with Soft Actor-Critic

Leonardo Luigi Pepe, 2157562

Abstract

This project investigates the swing-up and stabilization of the underactuated double pendulum in both Pendubot and Acrobot configurations. A classical hierarchical baseline is compared with reinforcement learning methods based on Soft Actor-Critic, including a hybrid SAC-LQR controller, a pure SAC policy, and a SAC policy fine-tuned with SNES. Results show that the hybrid approach provides the best overall performance, combining fast swing-up with reliable stabilization, while the Acrobot remains the more challenging task due to its unstable and highly variable training behavior.

1. Introduction

This report presents a Reinforcement Learning (RL) framework designed for the swing-up and stabilization of a robotic double pendulum. Specifically, the study focuses on two underactuated configurations of this system: the **Pendubot**, which is actuated at the first joint (base), and the **Acrobot**, actuated at the second joint (elbow) connecting the two links.

For each configuration, a classical baseline and three distinct RL strategies are evaluated. All three RL approaches are built upon the Soft Actor-Critic (SAC) algorithm:

1. **Hybrid SAC-LQR:** The SAC agent is trained to execute the swing-up maneuver, effectively driving the system into the Region of Attraction (RoA). Once inside the RoA, a Linear Quadratic Regulator (LQR) takes over to handle the local stabilization.
2. **End-to-End SAC:** The training phase is extended, allowing a single RL policy to autonomously manage both the nonlinear swing-up and the balancing phases.
3. **SAC + SNES Fine-tuning:** Building upon the End-to-End SAC policy, a Separable Natural Evolution Strategy (SNES) is applied to further refine and optimize the learned weights of the policy.

The primary objective of this study is to conduct a comparative analysis of these three methodologies across both robotic configurations to identify the most efficient approach. Performance is evaluated based on three key metrics: training time, energy consumption (control effort), and control accuracy.

Finally, the report concludes with a broader comparison between the proposed RL-based methods and classical nonlinear control techniques. This section will highlight the respective pros and cons, address critical aspects of the implementation, and propose potential solutions and avenues for future developments.

2. System Description

2.1. System Dynamics and Energy

The state-space representation and energy definitions are derived from the dynamic model documented by the DFKI Underactuated Robotics Lab (DFKI RIC Underactuated Lab, 2022). The double pendulum state is defined by joint angles $\mathbf{q} = [q_1, q_2]^T$ and velocities $\dot{\mathbf{q}}$.

The nonlinear equations of motion are given by:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}\big(\mathbf{D}\mathbf{u} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) - \mathbf{F}(\dot{\mathbf{q}})\big) \quad (1)$$

where $\mathbf{u} = [u_1, u_2]^T$ are the input torques. Here, $\mathbf{M}(\mathbf{q})$ is the symmetric positive-definite mass matrix, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ is the Coriolis/centrifugal matrix, $\mathbf{G}(\mathbf{q})$ contains the gravity terms, and $\mathbf{F}(\dot{\mathbf{q}})$ models the joint friction.

The actuation matrix $\mathbf{D}$ defines the underactuated nature of the specific configuration. For the base-actuated Pendubot and the elbow-actuated Acrobot, it is respectively:

$$\mathbf{D}_{\text{pendu}} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{D}_{\text{acro}} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \quad (2)$$

Finally, the total mechanical energy $E(\mathbf{q}, \dot{\mathbf{q}})$, calculated as the sum of kinetic and potential energies, is continually tracked to shape the reward function during the swing-up phase.

2.2. Baseline Controllers

To evaluate the performance of the proposed RL architectures, a set of classical control baselines has been developed for both the Pendubot and Acrobot configurations. The

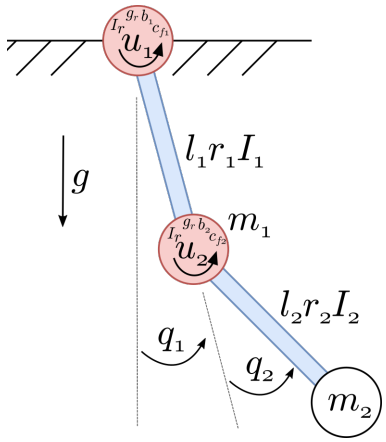

Figure 1. Schematic model of the double pendulum.

baseline strategies rely on a finite-state machine approach, managed by a custom hierarchical controller (*TripleController*), which transitions between different control laws based on the system's current state $\mathbf{x} = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T$.

The control architecture is divided into three distinct phases for both systems, leading the pendulum from the downward stable equilibrium to the upward unstable equilibrium $\mathbf{x}_{goal} = [\pi, 0, 0, 0]^T$:

1. **Swing-up Phase:** When the system is far from the target equilibrium, a dedicated swing-up controller injects energy into the system.
   - For the **Pendubot**, a custom *Progressive Feedback Swing Controller* is utilized, which employs a PD-like action (with $K_p = 15.0, K_d = 1.85$) to systematically build momentum.
   - For the **Acrobot**, a *Random Exploration Controller* is deployed. It generates bounded exploratory torques ($|u| \leq 10$ Nm) filtered at 50 Hz to excite the system dynamics until the links approach the upward position.
2. **Catching Phase (LQR):** Once the system enters a loosely defined Region of Attraction (RoA)—specifically when the positional errors are within a predefined threshold (e.g., $|\Delta q_i| < 0.2$ rad) and velocities are bounded—the system switches to a Linear Quadratic Regulator (LQR). This initial LQR is tuned with aggressive cost matrices $(\mathbf{Q}, \mathbf{R})$ to abruptly dampen the residual kinetic energy from the swing-up phase and pull the system towards the exact upright equilibrium.
3. **Stabilization Phase (Robust LQR):** When the system achieves the final goal condition ($|\Delta q_i| < 0.1$ rad and $|\dot{q}_i| < 0.3$ rad/s), the controller locks into a *Stabilization Phase*. A secondary, more conservatively tuned robust LQR takes over. This final controller is designed to maintain the unstable equilibrium indefinitely, minimizing the control effort and preventing excessive oscillations around the setpoint.

Both simulations are run at a control frequency of 1000 Hz ($dt = 0.001$ s) using a Runge-Kutta integrator over a 20-second horizon. The torque outputs are strictly saturated according to the physical limits of each respective actuator.

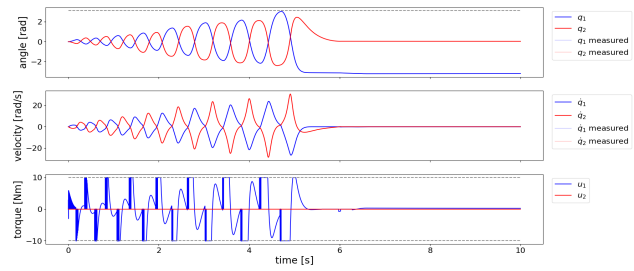

Figure 2. Pendubot baseline timeseries plot

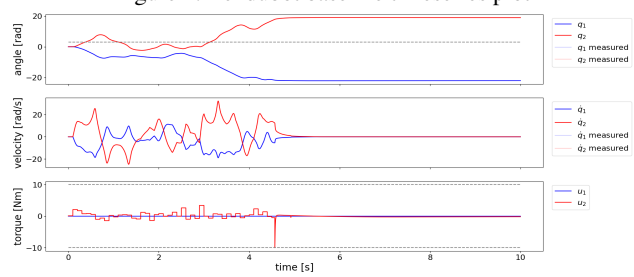

Figure 3. Acrobot baseline timeseries plot

3. Soft Actor-Critic

For the reinforcement learning framework, we selected the Soft Actor-Critic (SAC) algorithm. This choice is primarily driven by the continuous nature of both the state space (positions and velocities) and the action space (actuator torques) inherent in the double pendulum system. SAC is an off-policy actor-critic algorithm that excels in these scenarios by providing high sample efficiency and robustness.

3.1. Entropy-Augmented Objective

Unlike standard RL algorithms that aim solely to maximize the expected cumulative reward, SAC incorporates **entropy regularization**. The agent is trained to maximize a trade-off between the expected return and the policy's entropy H, which measures the randomness of action selection. The optimal policy π^* is defined as:

$$\pi^* = \arg\max_\pi \mathbb{E}_{\tau\sim\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot|s_t))\right)\right] \tag{3}$$

where α is the temperature parameter. This temperature serves a dual purpose: it encourages exploration by preventing the policy from prematurely collapsing onto a single suboptimal action and improves robustness by seeking a wide range of effective behaviors.

3.2. Architecture and Learning

SAC utilizes a multi-network architecture to improve stability and reduce overestimation bias:

- **The Actor:** Approximates a stochastic policy π_θ, mapping states to a probability distribution over actions.
- **The Critic:** Maintains two independent Q-functions, Q_{ϕ_1} and Q_{ϕ_2}, to estimate the value of state-action pairs.

The Q-networks are updated by minimizing the Bellman residual, where the target y is computed using the minimum of two target Q-networks to mitigate bias:

$$y = r + \gamma(1-d)\left(\min_{j=1,2} Q_{\text{targ},j}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s')\right) \tag{4}$$

with $\tilde{a}' \sim \pi_\theta(\cdot|s')$. This mathematical framework allows SAC to balance exploitation and exploration effectively, making it well-suited for the complex, nonlinear dynamics of the Pendubot and Acrobot.

4. Optimization Strategies

4.1. LQR Stabilization within the RoA

Relying purely on a stochastic SAC policy for precise stabilization often results in steady-state errors and control chattering. To address this, the control architecture transitions to a Linear Quadratic Regulator (LQR) once the system enters the Region of Attraction (RoA) of the upward equilibrium (DFKI RIC Underactuated Lab, 2023b).

The LQR controller is designed by linearizing the double pendulum dynamics around the target unstable equilibrium $\mathbf{x}_d = [\pi, 0, 0, 0]^T$. Solving the continuous-time Algebraic Riccati Equation for the linearized system yields the optimal feedback law $\tilde{\mathbf{u}} = -\mathbf{K}\tilde{\mathbf{x}}$, where $\tilde{\mathbf{x}} = \mathbf{x} - \mathbf{x}_d$ is the state error and $\mathbf{K}$ is the optimal gain matrix derived from the cost weights $\mathbf{Q}$ and $\mathbf{R}$.

Because direct computation of the exact RoA is analytically intractable for highly nonlinear systems, it is conservatively estimated using the LQR cost-to-go as a quadratic Lyapunov function, mathematically defined as a sublevel set:

$$\mathcal{B}_{est} = \{\tilde{\mathbf{x}} \mid \tilde{\mathbf{x}}^T \mathbf{S} \tilde{\mathbf{x}} \leq \rho\} \tag{5}$$

where $\mathbf{S}$ is the Riccati solution matrix and ρ is a specific bounding threshold. During execution, as soon as the state error satisfies this condition, the framework immediately terminates the SAC exploration and engages the LQR to achieve exact stabilization.

4.2. Policy Refinement via SNES

To further improve the robustness and performance of the trained SAC agents, we applied a fine-tuning phase using the Separable Natural Evolution Strategy (SNES), drawing direct inspiration from Calì et al. (Calì et al., 2023).

SNES is particularly effective for continuous parameter spaces with varying scales. By maintaining an independent step size for each policy weight dimension, this strategy combines the robust exploration of evolutionary algorithms with an adaptive step-size mechanism, achieving faster convergence and lower computational complexity compared to traditional CMA-ES architectures.

5. Environment design

5.1. Reward Function

The reward function is purely based on an energy-shaping approach, designed to inject energy into the system without causing uncontrollable spinning. The state space is partitioned into two regions based on the vertical position y of the end-effector relative to a defined threshold y_{th}.

The continuous step reward $R(s,a)$ is defined as:

$$R(s,a) = \begin{cases} V(s) - \beta T(s) & \text{if } y \geq y_{th} \\ V(s) - w_u u^2 - w_{\dot{q}} \|\dot{\mathbf{q}}\|^2 & \\ \quad - w_{\Delta u}|\Delta a| - \mathcal{P}_{stall}(s) & \text{otherwise} \end{cases} \tag{6}$$

Where:

- $V(s)$ and $T(s)$ are the potential and kinetic energies.
- In the upper phase ($y \geq y_{th}$), the kinetic energy is penalized by β to softly decelerate the pendulum as it approaches the unstable equilibrium.
- In the lower phase, the agent is regularized by penalizing the squared normalized action u^2 (with weight w_u), the squared joint velocities $\|\dot{\mathbf{q}}\|^2$ (with weight $w_{\dot{q}}$), and the action variation $|\Delta a| = |u_t - u_{t-1}|$ (with weight $w_{\Delta u}$) to ensure smooth torque commands.
- $\mathcal{P}_{stall}(s)$ is an anti-stall penalty exclusively active for the Acrobot. Because the Acrobot struggles to build momentum solely from the elbow joint, this term prevents the policy from converging to local minima (e.g., hovering at low velocities):

$$\mathcal{P}_{stall}^{acro}(s) = \frac{1}{10^{-4} + \|\dot{\mathbf{q}}\|^2} + \exp\left(-\frac{y}{y_{th}}\right) \tag{7}$$

  For the Pendubot, this term is null ($\mathcal{P}_{stall}^{pendu}(s) = 0$).

Terminal Bonus: If the agent successfully drives the system into the LQR Region of Attraction, an episode-ending bonus is awarded. The bonus scales with the state-alignment quality. For the **Pendubot**:

$$R_{term}^{pendu} = 1500 \cdot \big(-\cos(q_1) + \cos(q_2) + \Delta q_1 + \Delta q_2 \big) \quad (8)$$

For the **Acrobot**, additional velocity regularizations are included:

$$R_{term}^{acro} = 4 \cdot \Big(-\cos(q_1) + \cos(q_2) - |\Delta q_1| - |\Delta q_2| + 1 - \frac{\dot{q}_1^2}{100} - \frac{\dot{q}_2^2}{100} \Big) \quad (9)$$

where Δq_i is the angular error relative to the target equilibrium.

Table 1. Reward function hyperparameters for Pendubot and Acrobot.

Parameter	Symbol	Pendubot	Acrobot
Height Threshold [m]	y_{th}	0.30	0.35
Kinetic Penalty	β	1.0	0.9
Action Penalty	w_u	0.02	0.015
Velocity Penalty	$w_{\dot{q}}$	0.05	0.05
Action Variation Penalty	$w_{\Delta u}$	0.15	0.15
Terminal Multiplier	K_{term}	1500.0	4.0

5.2. Termination and Reset Strategy

Episodes terminate when the system reaches the RoA (success) or enters unsafe states ($|q_1| > 4\pi$ for Acrobot). The initial state is sampled from a curriculum distribution: 50% at rest, 24% mid-swing, 24% near the top, 2% at goal. This mitigates reward sparsity during early training.

6. Experimental Setup

This section details the simulation environment and the hyperparameters used for both the reinforcement learning training and the subsequent evolutionary refinement. All experiments share a common framework to ensure a consistent comparison across different control strategies.

6.1. Training Configuration

All experiments were conducted using a synchronized parallel training setup with 50 concurrent environments. The agents were trained using the Soft Actor-Critic (SAC) algorithm with a learning rate of 0.001. The control frequency was set to 100 Hz (sampling time $\Delta t = 0.01$ s). Evaluation was performed every $10,000/N_{envs}$ steps, averaging the performance over 10 evaluation episodes.

The task constraints and computational effort for each robot configuration are summarized in Table 2.

Table 2. Task constraints and training computational effort.

Feature	Pendubot	Acrobot
Ep. Duration	1.0 s (100 st.)	5.0 s (500 st.)
Torque Limit	10.0 Nm (J1)	4.0 Nm (J2)
Steps (SAC+LQR)	$\sim 170 \times 10^6$	$\sim 100 \times 10^6$
Time (SAC+LQR)	$\sim$ 10–12 h	$\sim$ 8–9 h
Steps (SAC/SNES)	$\sim 300 \times 10^6$	$\sim 200 \times 10^6$
Time (SAC/SNES)	$\sim$ 24 h	$\sim$ 16 h

6.2. Training Dynamics and Convergence

Analyzing the learning curves provides valuable insights into the complexity of the tasks and the behavior of the SAC algorithm.

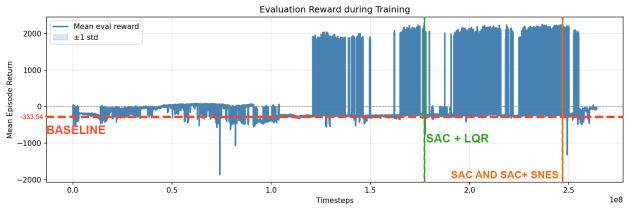

Figure 4. Pendubot evaluation curve training

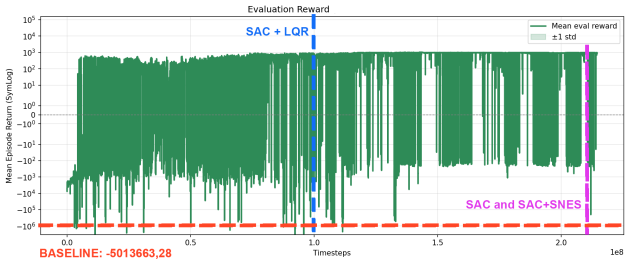

Figure 5. Acrobot evaluation curve during training. The Y-axis employs a symmetric logarithmic scale (symlog) to accommodate the extreme negative penalties incurred during early exploration, which would otherwise compress the convergence region. The red dashed line represents the classical baseline performance.

The Pendubot learning curve (Figure 4) shows a prolonged exploration phase (100M steps), then breakthrough at 1.2×10^8The Acrobot (Figure 5) exhibits extreme negative spikes during early exploration due to uncontrolled spinning; symlog scale is used for readability. Vertical lines mark checkpoints for different policy extractions (SAC+LQR vs. pure SAC/SNES).

6.3. SNES Optimization Parameters

For the policy refinement stage, the weights of the pre-trained SAC policies were optimized using the Separable Natural Evolution Strategy (SNES). The optimization process was distributed using the Ray framework to manage parallel population evaluations. The hyperparameters for the evolutionary process are reported in Table 3.

The increased population size and generation count for the Pendubot reflect the higher complexity of its swing-up ma-

Table 3. Hyperparameters for SNES policy refinement.

Hyperparameter	Pendubot	Acrobot
Population Size (N_{pop})	40	16
Generations	1500	500
Initial Std. Deviation (σ_{init})	0.01	0.01
Fitness Eval. Episodes	10	10

neuver compared to the Acrobot, necessitating a more extensive exploration of the policy weight space to achieve fine-tuned stabilization.

7. Results

The performance of the proposed controllers is evaluated using a comprehensive set of metrics, including swing-up time, energy consumption, and torque smoothness, following the standardized evaluation framework established for the AI Olympics challenge (DFKI RIC Underactuated Lab, 2023a).

Table 4. Performance metrics for the Pendubot swing-up task.

Metric	Baseline	SAC	SAC_SNES	SAC_LQR
Success	Yes	Yes	Yes	Yes
Time [s]	5.508	1.010	1.010	0.955
Pos. Err. [°]	5.07	7.68	6.72	5.19
Jitter [rad/s]	0.004	0.006	0.007	0.002
Energy [J]	243.67	22.71	22.62	24.85
Max τ [Nm]	10.00	9.39	9.41	10.00
Int. τ [Nm]	27.89	6.28	6.04	5.23
τ Cost [N^2m^2]	211.15	32.72	31.25	29.38
τ Smoot. [N^2m^2]	2.327	2.071	1.745	0.058
Vel. Cost [m^2/s^2]	1072.70	67.52	67.56	68.05

Table 5. Performance metrics for the Acrobot swing-up task.

Metric	Baseline	SAC	SAC_SNES	SAC_LQR
Success	Yes	Marg.	Yes	Yes
Time [s]	4.625	1.400	2.240	1.766
Pos. Err. [°]	7.38	33.62	4.82	1.27
Jitter [rad/s]	0.017	0.052	0.025	0.030
Energy [J]	39.77	10.83	11.95	9.49
Max τ [Nm]	10.00	3.60	3.88	10.00
Int. τ [Nm]	4.80	5.57	3.27	2.40
τ Cost [N^2m^2]	5.95	4.67	6.27	3.97
τ Smoot. [N^2m^2]	0.141	0.120	0.307	0.093
Vel. Cost [m^2/s^2]	1111.68	106.10	131.68	87.55

Note on Marginal Success: A result is classified as "Marginal" when the controller achieves a stable local equilibrium but fails to reach the global vertical optimum. In the Acrobot SAC case, the high Final Position Error (33.62°) is due to a torque-gravity deadlock: once the initial kinetic energy is dissipated, the controller's residual torque is insufficient to overcome gravitational and frictional resistance, resulting in a stable but misaligned steady state.

Understanding the Evaluation Metrics

To accurately evaluate the controllers' real-world applicability and decouple sheer speed from stabilization quality, three specific metrics are introduced:

- **Swing-up Time [s]:** First hitting time into the acceptable goal tolerance zone.
- **Final Pos Error [°]:** Root Mean Square Error (RMSE) of the joint angles relative to the perfect upright target during the final 0.5 seconds.
- **Final Jitter [rad/s]:** RMSE of the joint velocities over the final 0.5 seconds, quantifying the high-frequency "nervousness" of the steady-state control signal.

As evidenced by the quantitative results presented in the tables, the hybrid SAC+LQR configuration emerges as the most effective control strategy. Primarily, this architecture achieves the fastest swing-up maneuver while guaranteeing the most precise and robust stabilization phase around the target unstable equilibrium.

Furthermore, these performance and accuracy gains are attained with an overall energy expenditure that is comparable to—and in several aspects, significantly lower than—the purely RL-based baseline and the SNES fine-tuned approach. This efficiency is particularly highlighted by two critical control metrics:

- **Torque Cost (Control Effort):** This metric quantifies the total mechanical and electrical energy consumed by the actuators throughout the episode. It is typically calculated as the sum of the squared torque signals over time ($\sum u_t^2$). A lower torque cost indicates a more energy-efficient policy that achieves the goal without unnecessarily saturating the motors.
- **Torque Smoothness:** This metric evaluates the high-frequency variations, or "chattering," in the control signal. It is measured by the sum of the squared differences between consecutive torque commands ($\sum(u_t - u_{t-1})^2$). A high degree of smoothness (represented by a lower metric value) is crucial for real-world hardware deployment, as it minimizes mechanical wear, prevents actuator overheating, and avoids exciting unmodeled structural dynamics.

The superiority of the SAC+LQR approach in these specific metrics highlights a fundamental advantage of the hybrid architecture: while the RL policy explores and resolves the highly nonlinear swing-up phase, delegating the final balance task to an LQR completely eliminates the continuous, noisy micro-corrections typical of stochastic neural network outputs, resulting in a much cleaner, safer, and efficient control signal.

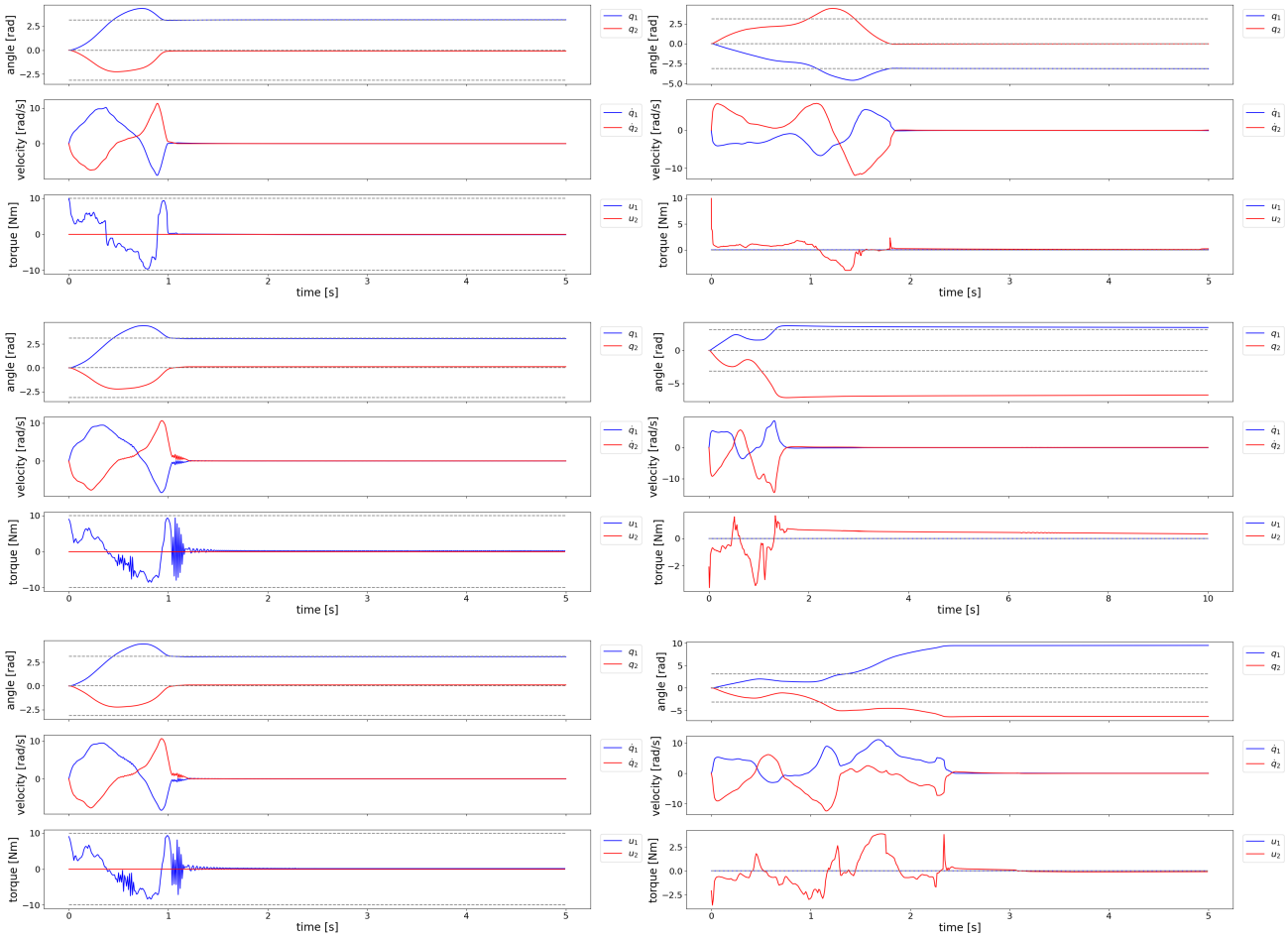

Figure 6. Timeseries performance comparison. **Left column:** Pendubot. **Right column:** Acrobot. **Top row:** Hybrid SAC+LQR controller. **Middle row:** Pure SAC policy. **Bottom row:** SAC policy fine-tuned with SNES.

8. Conclusion and Future Work

The experimental results demonstrate the successful application of the proposed methodologies, achieving the desired performance metrics and control objectives for both the Pendubot and Acrobot configurations. Overall, this project provided a highly valuable, hands-on introduction to the integration of Reinforcement Learning (RL) within the domain of robotics and underactuated control systems.

Despite the substantial initial time investment required for environment engineering, reward shaping, and the inherently lengthy training phases, it is evident that RL serves as a powerful complementary tool to classical control strategies. As familiarity with these computational frameworks grows, hybrid architectures like the one presented can effectively bridge the gap between traditional control theory and modern machine learning techniques.

Furthermore, the extensive training time highlights an inherent critical aspect of model-free algorithms in this specific domain. For systems governed by well-defined but highly nonlinear dynamics like the double pendulum, model-based RL approaches, such as MC-PILCO (Sinigaglia et al., 2024a), tend to be significantly more sample-efficient than Soft Actor-Critic. While SAC requires millions of interactions to implicitly learn the system's physics, it remains highly advantageous in other control contexts where analytical modeling is intractable or highly discontinuous. Brief examples include legged locomotion over unpredictable and rough terrain, or the dexterous robotic manipulation of soft, deformable objects, where SAC's stochastic exploration and model-free nature truly excel.

Future developments of this work may involve the sim-to-real transfer of the trained policies to physical hardware, allowing for the empirical validation of the robustness and the zero-shot generalization capabilities of the hybrid SAC-LQR architecture.

References

Achiam, J. Soft actor-critic — OpenAI Spinning Up documentation, 2018. URL https://spinningup.openai.com/en/latest/algorithms/sac.html. Accessed: 2024-05-22.

Åström, K. J., Furuta, K., Iwashiro, M., and Hoshino, T. Energy based strategies for swinging up a double pendulum. In *Proceedings of the 14th IFAC World Congress*, Beijing, China, 1999. Lund Institute of Technology and Tokyo Institute of Technology.

Calì, M., Sinigaglia, A., Turcato, N., Carli, R., and Susto, G. A. Ai olympics challenge with evolutionary soft actor critic, 2023. AI Olympics at IJCAI Challenge Report.

DFKI RIC Underactuated Lab. The double pendulum system - dynamics. https://dfki-ric-underactuated-lab.github.io/double_pendulum/dynamics.html, 2022. Accessed: April 2026.

DFKI RIC Underactuated Lab. Ai olympics at ijcai 2023 - double pendulum challenge. https://ai-olympics.dfki-bremen.de/aiolympics-at-ijcai-2023/, 2023a. Accessed: April 2026.

DFKI RIC Underactuated Lab. Linear quadratic regulator (lqr) — double pendulum 1.0.0 documentation. https://dfki-ric-underactuated-lab.github.io/double_pendulum/control.policy.lqr.html, 2023b. Accessed: April 2026.

Sinigaglia, A., Calì, M., and Marcuzzi, F. Solving the swing-up and balance task for the acrobot and pendubot with reinforcement learning and energy-based control, 2024a. URL https://arxiv.org/abs/2409.05811.

Sinigaglia, A., Calì, M., and Marcuzzi, F. Solving the swing-up and balance task for the acrobot and pendubot with reinforcement learning and energy-based control, 2024b. URL https://arxiv.org/abs/2409.01104.

Wiebe, F., Kumar, S., Shala, L. J., Vyas, S., Javadi, M., and Kirchner, F. Open source dual-purpose acrobot and pendubot platform: Benchmarking control algorithms for underactuated robotics. *IEEE Robotics & Automation Magazine*, 31(2):113–124, 2024. doi: 10.1109/MRA.2023.3341257.

Zhang, C., Sathuluri, A., and Zimmermann, M. Solving the swing-up and balance task for the acrobot and pendubot with sac. In *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pp. 2743–2750, 2021. doi: 10.1109/SMC52423.2021.9658763.